Module 14) Python – Collections, functions and Modules

3. Working with Lists

- **Iterating over a list using loops**

List iteration is the process of sequentially accessing each element in a list. Python provides multiple looping constructs for this purpose, each with specific characteristics and use cases.

Types of List Iteration

1. For Loops (Element-wise Iteration)

Primary Mechanism:
for item in list:

Characteristics:

- Accesses elements directly (not indices)
- Simple and Pythonic
- Read-only by default (cannot modify list structure during iteration)
- Creates a temporary reference to each element
- Time complexity: O(n) for complete traversal

2. For Loops (Index-based Iteration)

Primary Mechanism:
for i in range(len(list)):

Characteristics:

- Accesses elements via indices
- Allows modification of elements (list[i] = new_value)
- Enables access to neighboring elements
- More verbose than element-wise iteration
- Required when position information is needed

3. While Loops

Primary Mechanism:
while i < len(list):

Characteristics:

- More flexible termination conditions
- Can modify iteration index manually
- Risk of infinite loops if not properly managed
- Useful for complex traversal patterns
- Often requires manual index management

- **Sorting and reversing a list using sort(), sorted(), and reverse().**

Sorting Operations

1. sort() Method

Definition:
An in-place sorting method that modifies the original list.

Key Characteristics:

- Operates on the list object itself (mutates original list)
- Returns None (operation is performed in-place)

- Accepts two optional parameters:
    - key: Function to determine sort order
    - reverse: Boolean for descending order (default: False)
- Time complexity: O(n log n) in typical implementations
- Only works on homogeneous lists (elements must be comparable)

2. sorted() Function

Definition:
A built-in function that returns a new sorted list from any iterable.

Key Characteristics:

- Creates and returns a new sorted list
- Original iterable remains unchanged
- Accepts same parameters as sort():
    - key: Custom sort function
    - reverse: Sort direction flag
- Works with any iterable (not just lists)
- More versatile but uses more memory (creates new list)

Reversing Operations

1. reverse() Method

Definition:
An in-place method that reverses the order of list elements.

Key Characteristics:

- Modifies the original list directly
- Returns None (operation is performed in-place)
- Time complexity: O(n)
- Does not sort - simply reverses current order

- Works with heterogeneous lists (no comparison needed)

• Basic list manipulations: addition, deletion, updating, and slicing.

1. Addition Operations

a. Append (Single Element Addition)

- *Method:* list.append(element)
- *Behavior:* Adds element to the end of list
- *Time Complexity:* O(1) amortized
- *Modifies Original:* Yes
- *Returns:* None

b. Extend (Multiple Elements Addition)

- *Method:* list.extend(iterable)
- *Behavior:* Adds all elements from iterable to list end
- *Time Complexity:* O(k) for k new elements
- *Modifies Original:* Yes
- *Returns:* None

c. Insert (Positional Addition)

- *Method:* list.insert(index, element)
- *Behavior:* Inserts element before specified index
- *Time Complexity:* O(n)
- *Modifies Original:* Yes
- *Returns:* None

2. Deletion Operations

a. Remove (Value-Based)

- *Method:* list.remove(value)
- *Behavior:* Removes first occurrence of value
- *Time Complexity:* O(n)
- *Modifies Original:* Yes
- *Returns:* None
- *Error:* ValueError if value not found

b. Pop (Index-Based)

- *Method:* list.pop([index])
- *Behavior:* Removes and returns item at index (default last)
- *Time Complexity:* O(1) for end, O(n) otherwise
- *Modifies Original:* Yes
- *Returns:* Removed element
- *Error:* IndexError if list empty or index invalid

3. Updating Operations

a. Index Assignment

- *Method:* list[index] = new_value
- *Behavior:* Replaces element at specified index
- *Time Complexity:* O(1)
- *Modifies Original:* Yes
- *Error:* IndexError if index invalid

4. Slicing Operations

a. Basic Slice

- *Method:* list[start:stop:step]
- *Behavior:* Returns new list containing specified elements
- *Time Complexity:* O(k) for slice of size k
- *Modifies Original:* No
- *Default Values:* start=0, stop=len(list), step=1